



**RAJALAKSHMI
ENGINEERING COLLEGE**

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING LAB MANUAL**

CS23432 – Software Construction

(REGULATION 2023)

**RAJALAKSHMI ENGINEERING COLLEGE
Thandalam, Chennai-602015**

Name: LALITH SANTHOSH CHINNAKONDA RAJAGOPAL

RegisterNo: 241001122

Year / Branch / Section: II - IT

Semester: IV

Academic Year: 2025-2026

INDEX

S.No.	Date	Title
1.	21/01/2026	Azure Devops Environment Setup.
2.	28/01/2026	Azure Devops Project Setup and User Story Management.
3.	11/02/2026	Setting Up Epics, Features, And User Stories for Project Planning.
4.	25/02/2026	Sprint Planning.
5.	11/03/2026	Poker Estimation.
6.	18/03/2026	Designing Class and Sequence Diagrams for Project Architecture.
7.	01/04/2026	Designing Architectural and ER Diagrams for Project Structure.
8.	08/04/2026	Testing – Test Plans and Test Cases.
9.	15/04/2026	Load Testing and Pipelines.
10.	22/04/2026	GitHub: Project Structure & Naming Conventions.

EXPNO:1	AZUREDEVOPS ENVIRONMENT SETUP
---------	--------------------------------------

Aim:

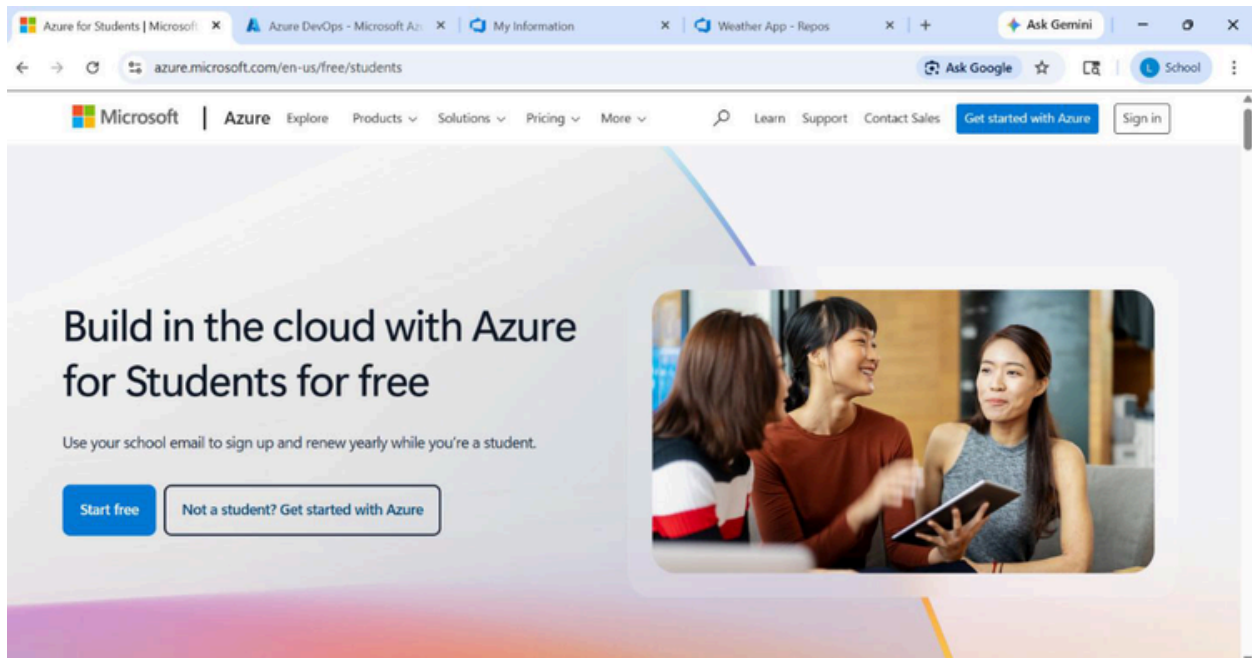
To set up and access the Azure DevOps environment by creating an organization through the Azure portal.

INSTALLATION

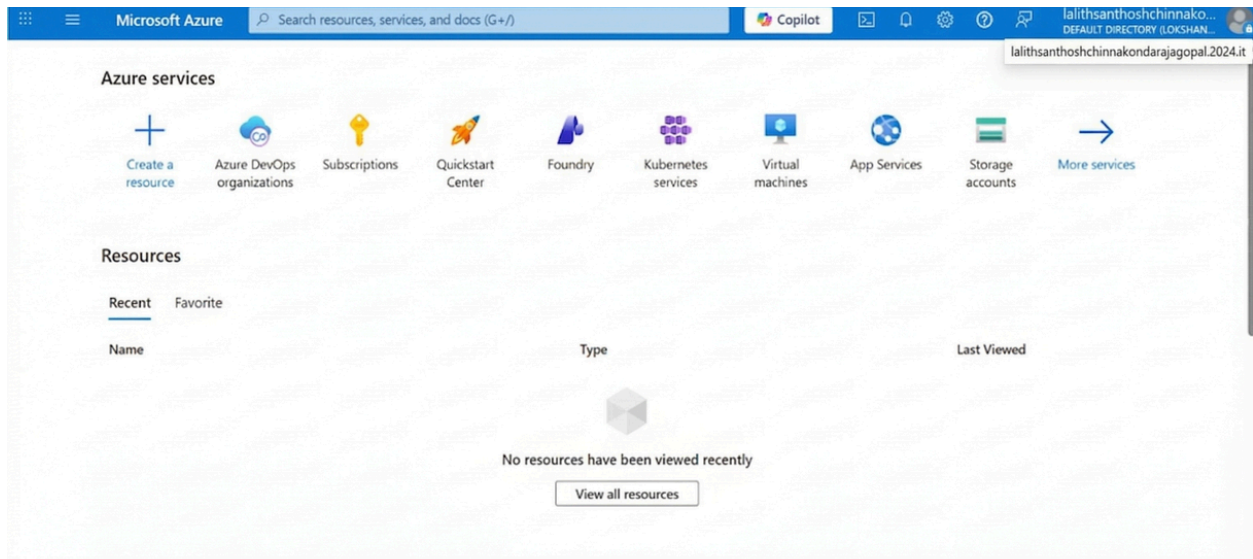
1. Open your web browser and go to the Azure website: <https://azure.microsoft.com/en-us/get-started/azure-portal>.

Sign in using your Microsoft account credentials.

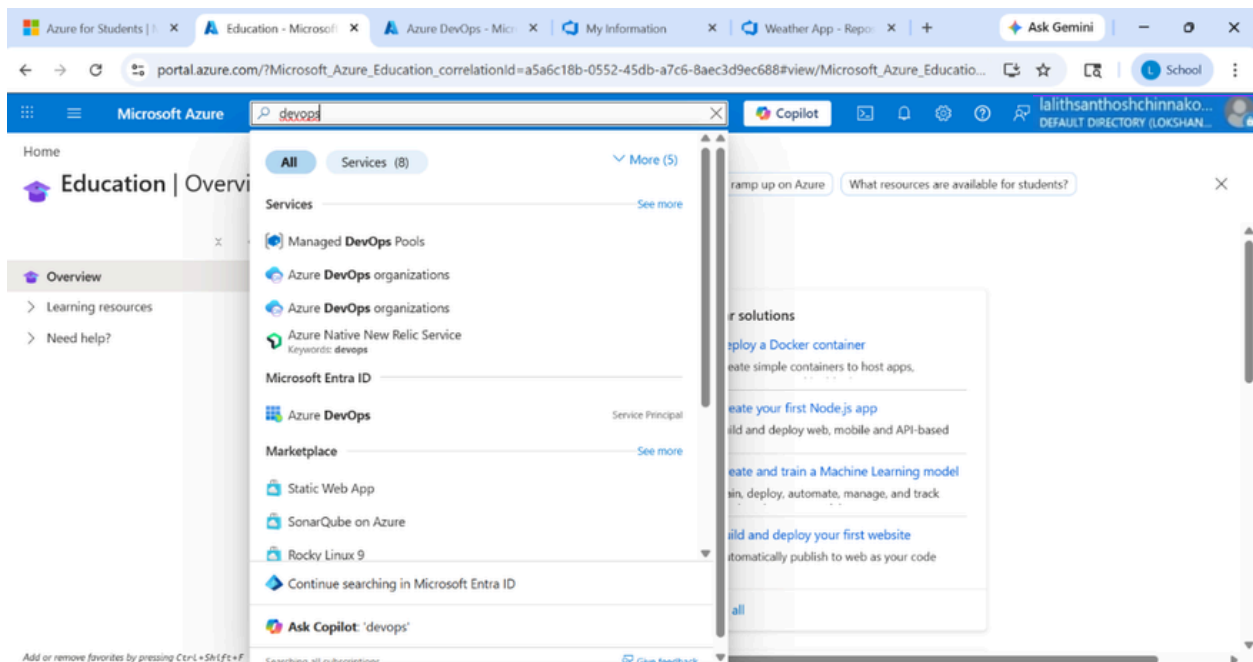
If you don't have a Microsoft account, you can create one here: <https://signup.live.com/?lic=1>



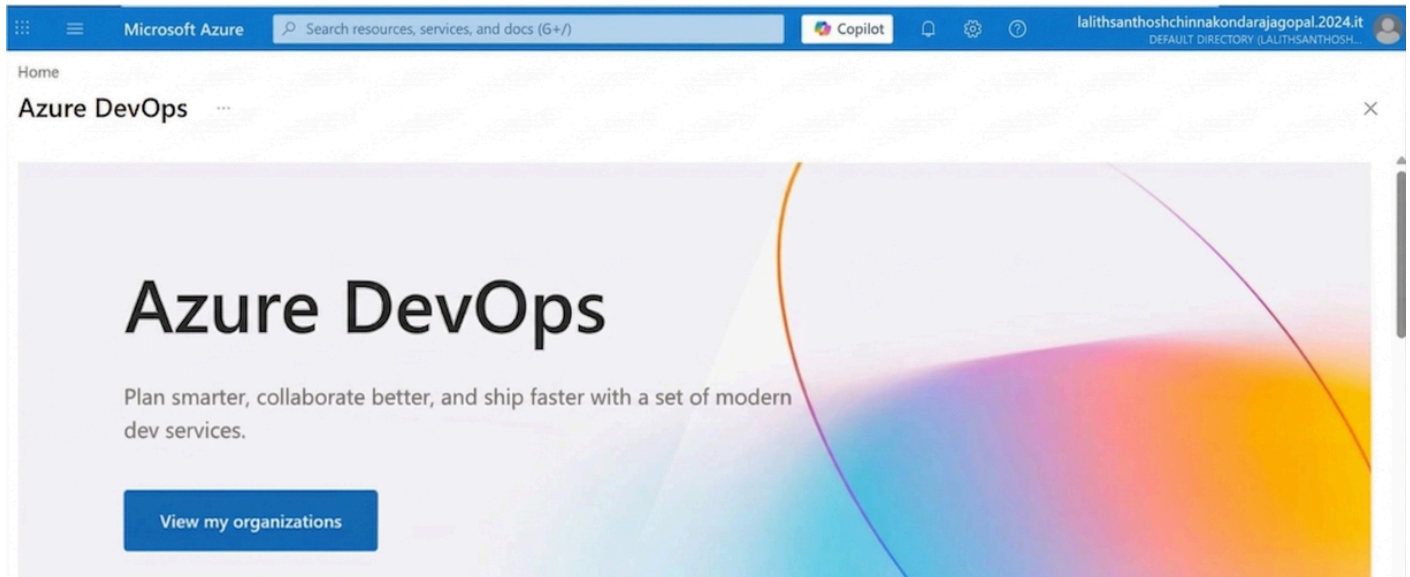
2. Azure home page



3. Open DevOps environment in the Azure platform by typing *Azure DevOps Organizations* in the search bar.



4. Click on the **My Azure DevOps Organization** link and create an organization and you should be taken to the Azure DevOps Organization Home page.



Result

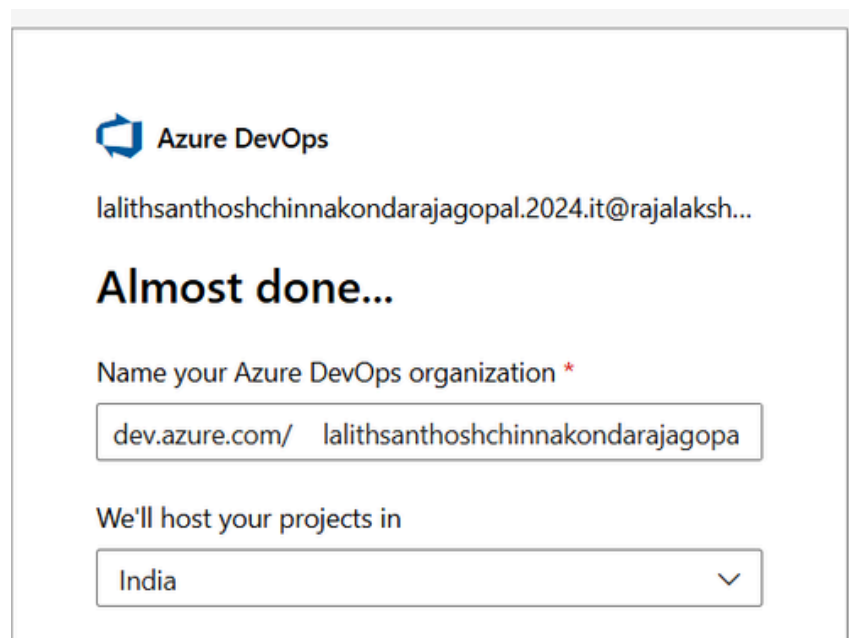
Successfully accessed the Azure DevOps environment and created a new organization through the Azure portal.

EXPNO:2	AZUREDEVOPS PROJECT SETUP AND USER STORY MANAGEMENT
---------	--

Aim:

To set up an Azure DevOps project for efficient collaboration and agile work management.

1.Create An Azure Account



The screenshot displays the final steps of creating an Azure DevOps organization. At the top, the Azure DevOps logo is visible. Below it, the user's email address is shown. The main heading is 'Almost done...'. There are two input fields: one for the organization name, which contains 'dev.azure.com/ lalithsanthoshchinnakondarajagopa', and another for the region, which is set to 'India'.

2.Create the First Project in Your Organization

- After the organization is set up, you'll need to create your first **project**. This is where you'll begin to manage code, pipelines, work items, and more.
- On the organization's **Home page**, click on the **New Project** button.
- Enter the project name, description, and visibility options:
 - Name:** Choose a name for the project (e.g., **LMS**).
 - Description:** Optionally, add a description to provide more context about the project.
 - Visibility:** Choose whether you want the project to be **Private** (accessible only to those invited) or **Public** (accessible to anyone).
- Once you've filled out the details, click **Create** to set up your first project.

Create new project

×

Project name *

Weather Apps

Description

Visibility

Private

Only people you give access to will be able to view this. Want to create a public project? [Try GitHub](#)

^ Advanced

Version control ?

Git

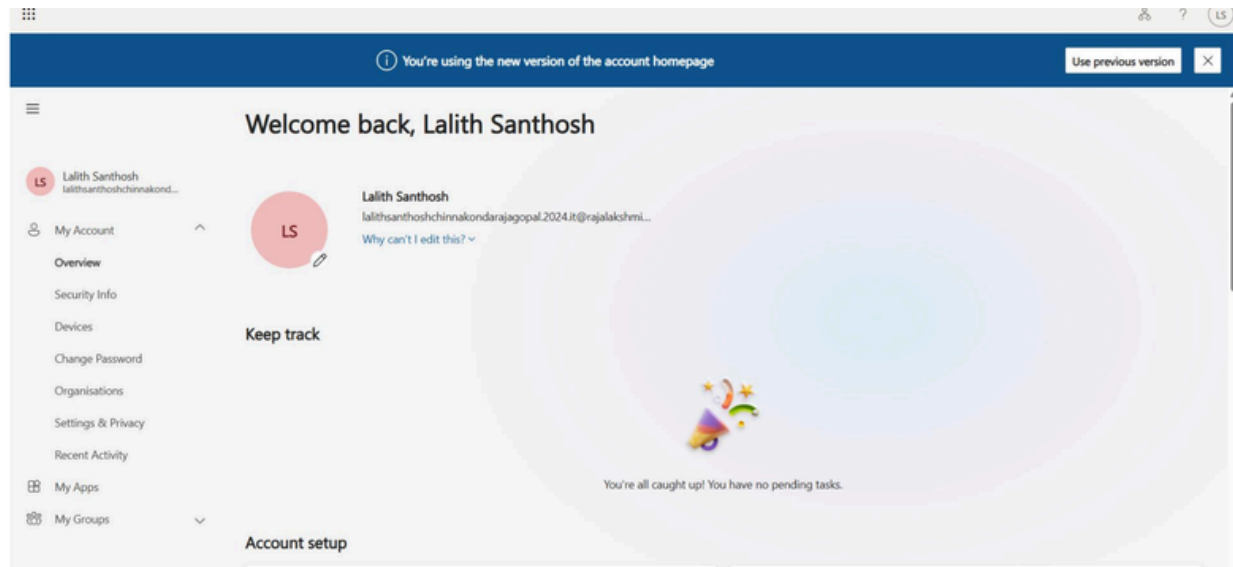
Work item process ?

Agile

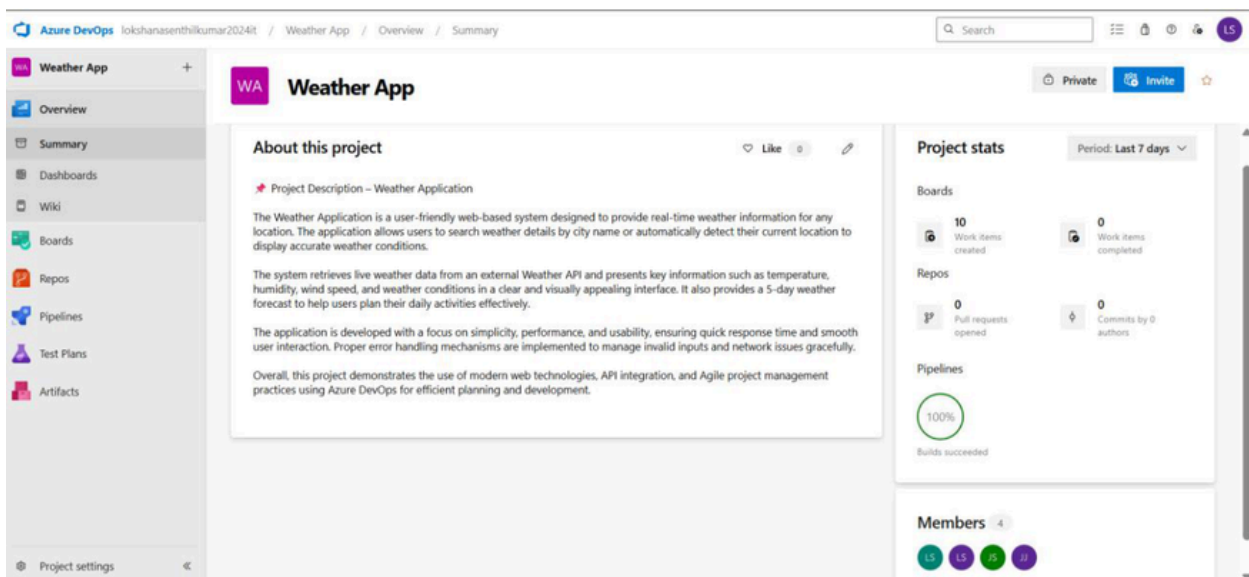
Cancel

Create

3.Once logged in, ensure you are in the correct organization. If you're part of multiple organizations, you can switch between them from the top left corner (next to your user profile). Click on the Organization name, and you should be taken to the Azure DevOps Organization Home page.



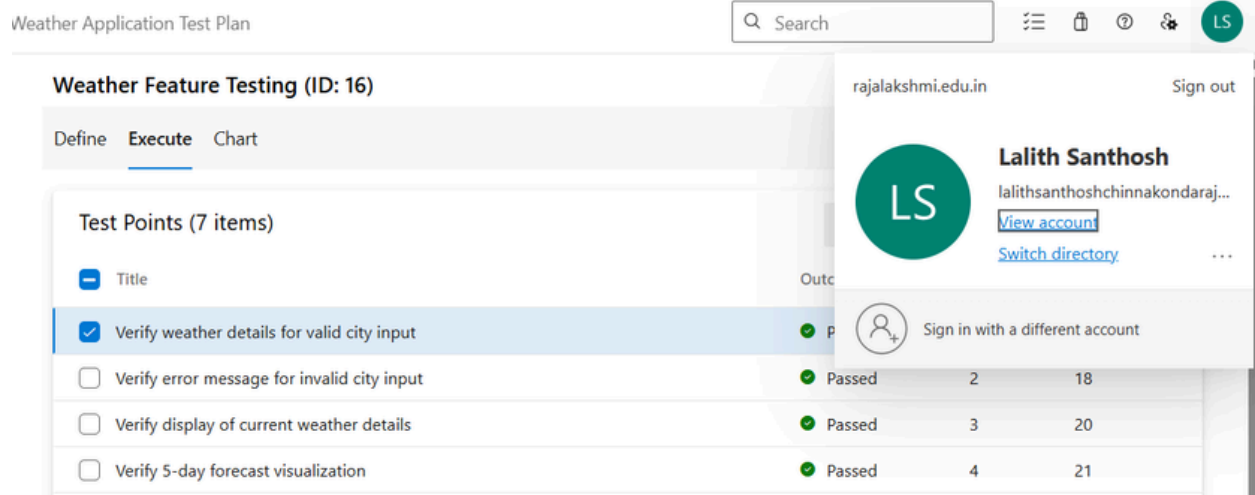
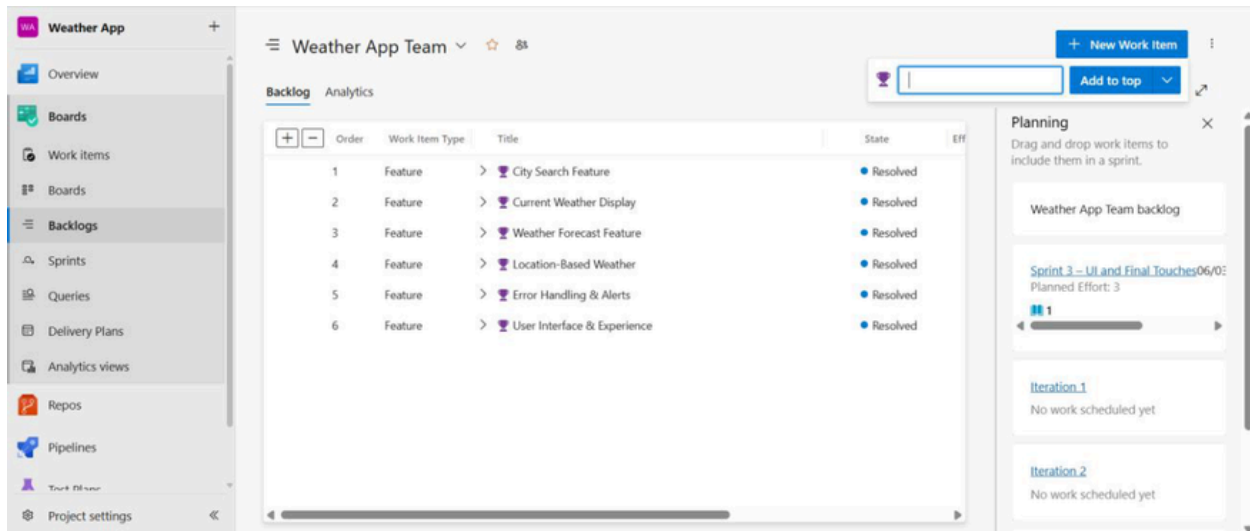
4. Project dashboard



5. To manage user stories:

a. From the **left-hand navigation menu**, click on **Boards**. This will take you to the main **Boards** page, where you can manage work items, backlogs, and sprints.

b. On the **work items** page, you'll see the option to **Add a work item** at the top. Alternatively, you can find a + button or **Add New Work Item** depending on the view you're in. From the **Add work item** dropdown, select **User Story**. This will open a form to enter details for the new User Story.



Result:

Successfully created an Azure DevOps project with user story management and agile workflow setup.

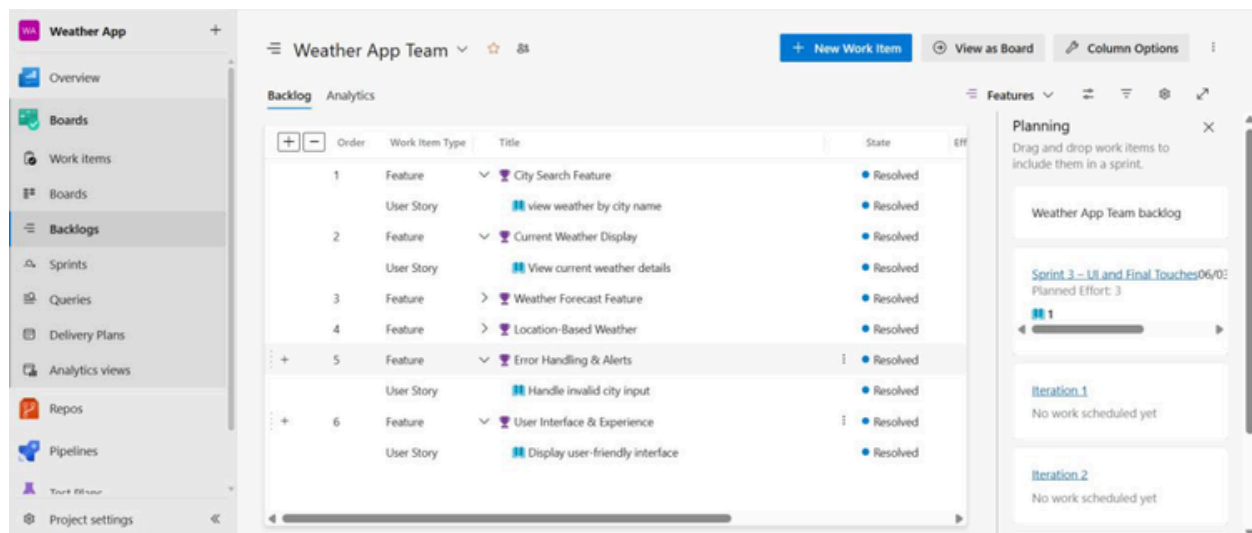
EXP NO: 3

SETTING UP EPICS, FEATURES, AND USER STORIES FOR PROJECT PLANNING

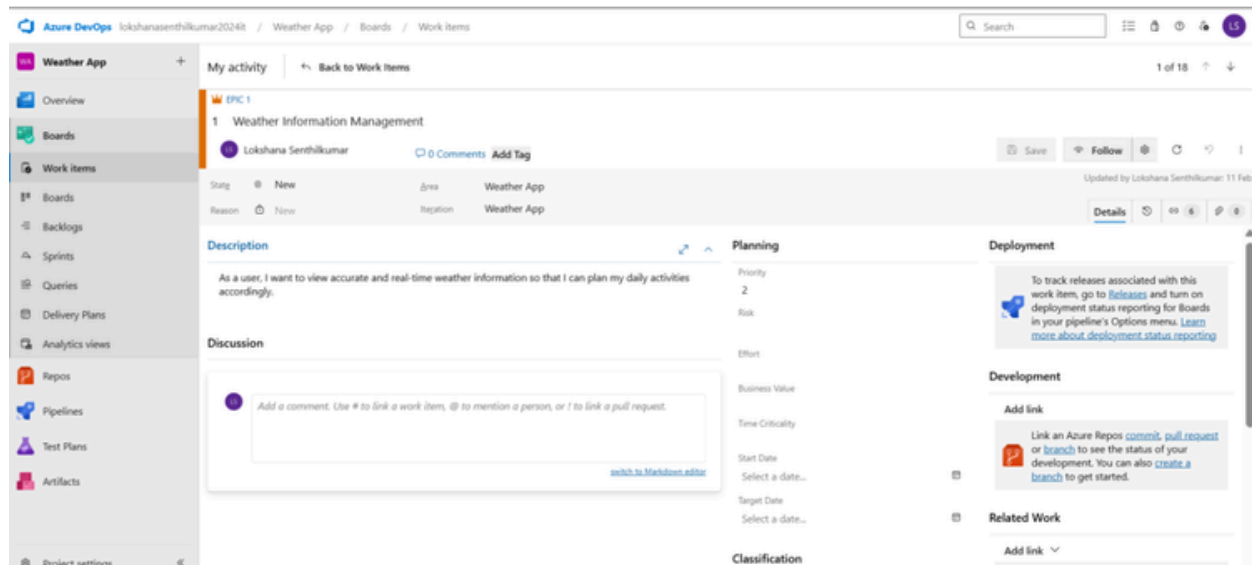
Aim:

To learn about how to create epics, user story, features, backlogs for your assigned project.

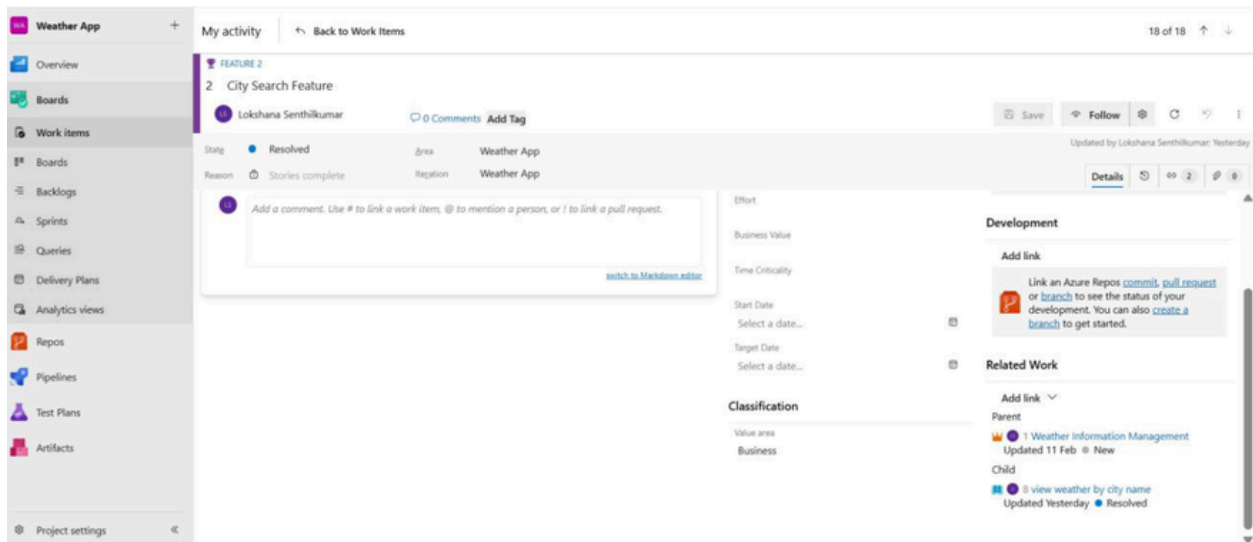
Create Epic, Features, User Stories, Task



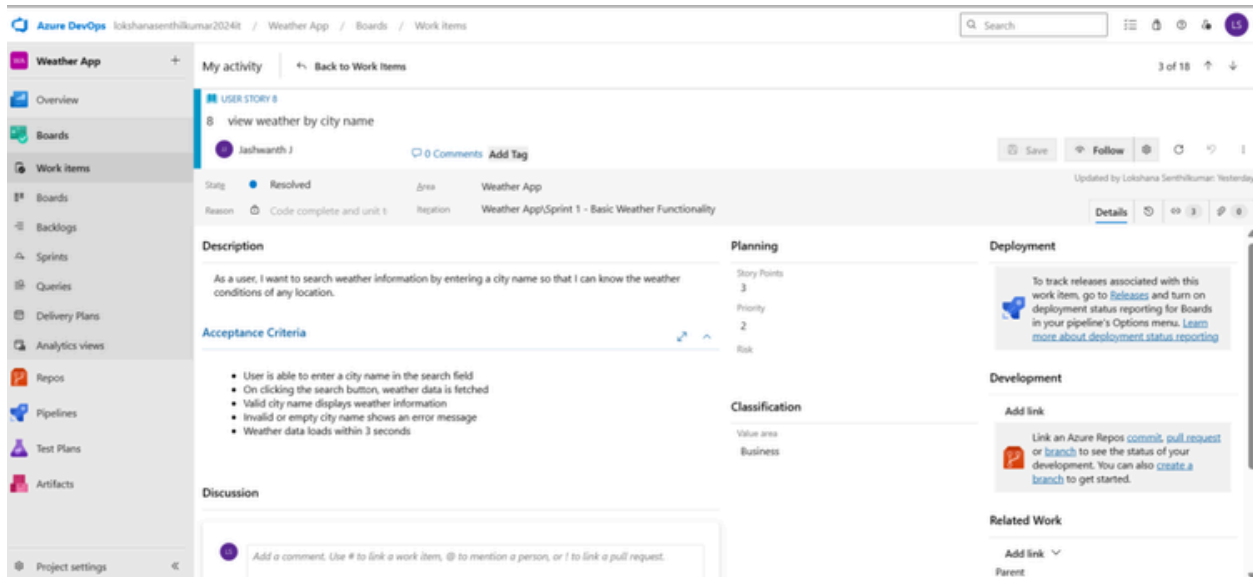
1.Fill in Epics



2.Fill in Features



3.Fill in User Story Details



Result:

Thus, the creation of epics, features, user story and task has been created successfully.

EXPNO: 4	SPRINT PLANNING
----------	-----------------

Aim:

To assign user story to specific sprint for the Weather App.

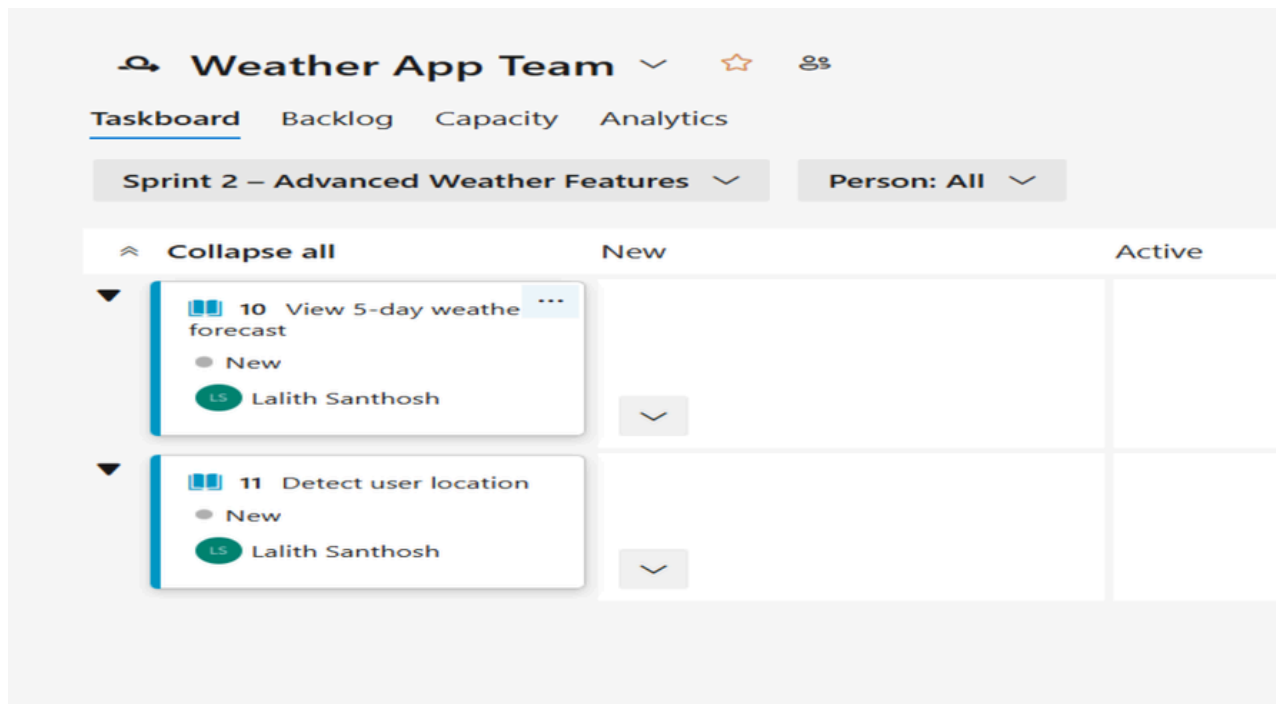
SprintPlanning

Sprint1

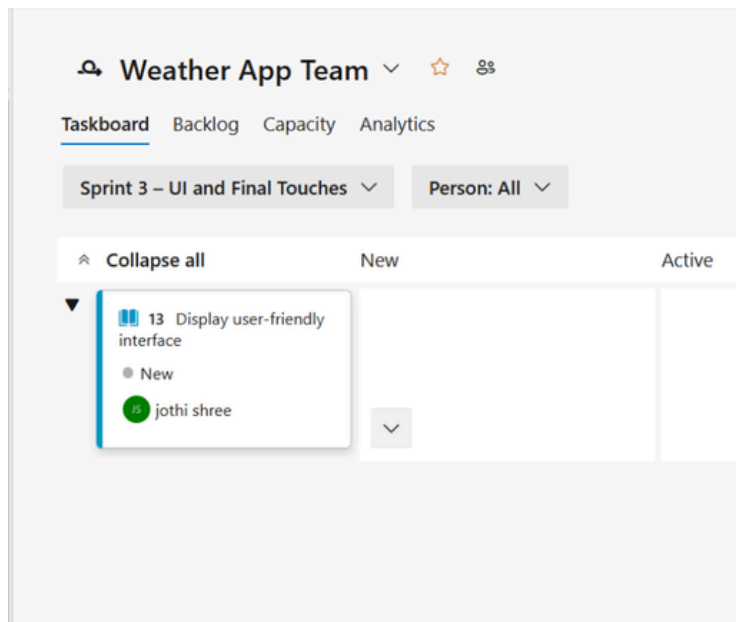
The screenshot shows the Jira Sprint Planning interface for the 'Weather App Team'. The board is set to 'Sprint 1 - Basic Weather Functionality' and filtered by 'Person: All'. The board is divided into three columns: 'Collapse all', 'New', and 'Active'. Three user stories are currently in the 'New' column:

- 8 view weather by city name**: Assigned to Jashwanth J.
- 9 View current weather details**: Assigned to Jashwanth J.
- 12 Handle invalid city input**: Assigned to jothi shree.

Sprint 2



Sprint 3



Result:

The Sprints are created for the Weather App.

EXP NO: 5

POKER ESTIMATION

Aim:

Create Poker Estimation for the user stories – Weather App Project.

PokerEstimation

The screenshot displays the Azure DevOps interface for a project named 'Weather App'. The left sidebar shows the navigation menu with options like Overview, Boards, Work items, Backlogs, Sprints, Queries, Delivery Plans, Analytics views, Repos, Pipelines, Test Plans, and Artifacts. The main area shows a 'My activity' view with a 'Back to Work Items' link. A specific User Story is selected: 'view weather by city name' (ID: 8), created by 'Jashwanth J'. The story is in the 'Resolved' state, with the reason 'Code complete and unit t'. The description states: 'As a user, I want to search weather information by entering a city name so that I can know the weather conditions of any location.' The acceptance criteria are listed as follows:

- User is able to enter a city name in the search field
- On clicking the search button, weather data is fetched
- Valid city name displays weather information
- Invalid or empty city name shows an error message
- Weather data loads within 3 seconds

The Planning section shows 'Story Points' as 3, 'Priority' as 2, and 'Risk' as Low. The Classification section shows 'Value area' as Business. The right sidebar contains sections for 'Deployment', 'Development', and 'Related Work'. The 'Deployment' section includes a link to 'Releases' and a note about deployment status reporting. The 'Development' section includes a link to 'Add link' and a note about linking to Azure Repos. The 'Related Work' section includes a link to 'Add link' and a note about linking to Parent.

Result:

The Estimation/Story Points is created for the project using Poker Estimation.

EXP NO: 6	DESIGNING CLASS AND SEQUENCE DIAGRAMS FOR PROJECT ARCHITECTURE
------------------	---

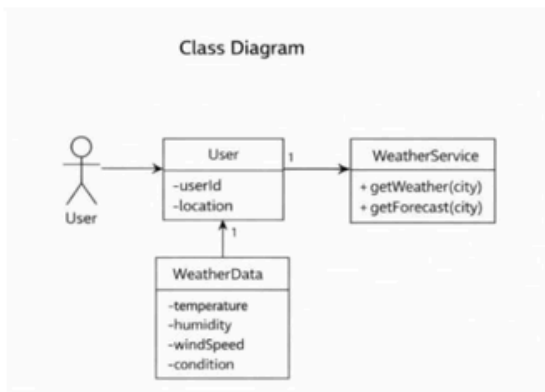
Aim:

To Design a Class Diagram and Sequence Diagram for the given Project.

6A. Class Diagram

Class Diagram

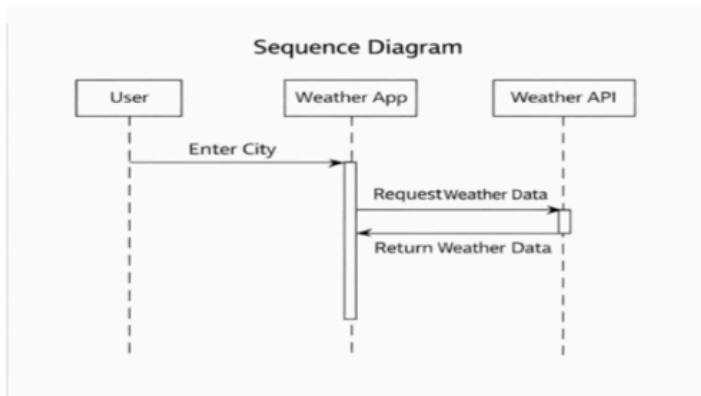
Lokshana Senthilkumar 3m ago



6B. Sequence Diagram

Sequence Diagram

Lokshana Senthilkumar 14m ago



Result:

The Class Diagram and Sequence Diagram is designed Successfully for the Weather App.

EXP NO: 7

DESIGNING ARCHITECTURAL AND ER DIAGRAMS FOR PROJECT STRUCTURE

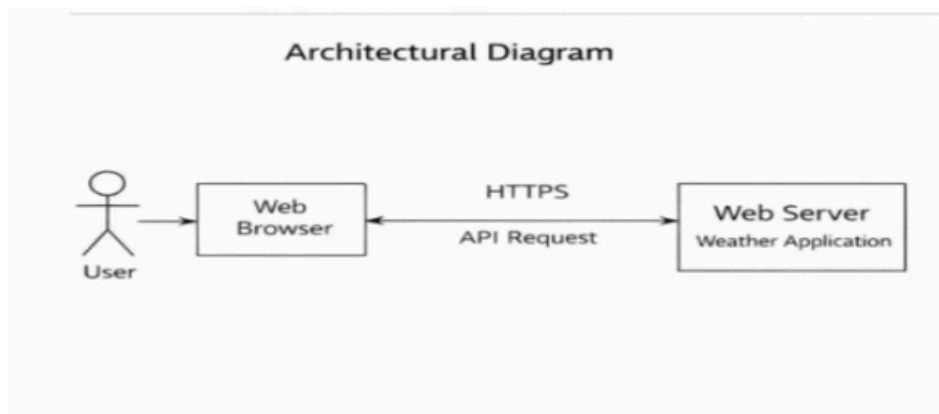
Aim:

To Design an Architectural Diagram and ER Diagram for the given Project.

7A. Architectural Diagram

Architectural Diagram

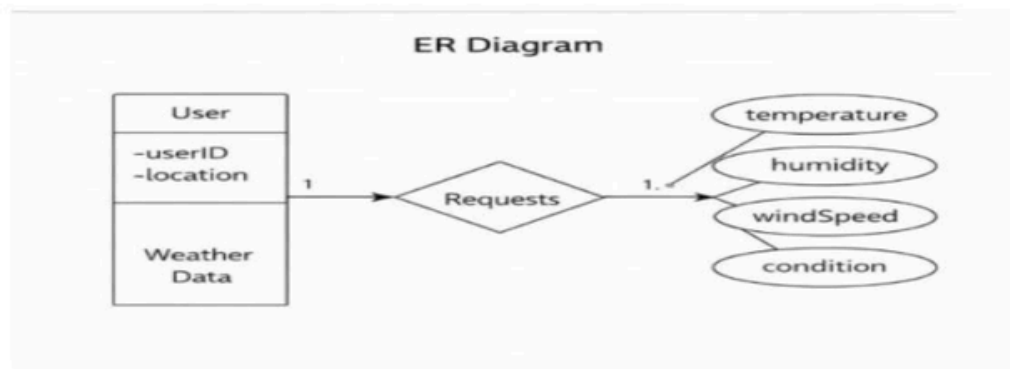
Lokshana Senthilkumar 20m ago



7B.ER Diagram

ER Diagram

Lokshana Senthilkumar 20m ago



Result:

The Architecture Diagram and ER Diagram is designed Successfully for the Weather App.

EXP NO: 8

TESTING – TEST PLANS AND TEST CASES

Aim:

Test Plans and Test Case and write two test cases for at least five user stories showcasing the happy path and error scenarios in azure DevOps platform.

Test Planning and Test Case

Test Case Design Procedure

1. Understand Core Features of the Application

- User Location Detection (GPS / Manual city search)
- Viewing Current Weather Conditions
- 7-Day / Hourly Weather Forecast
- Search Weather by City
- Error Handling (invalid city, no internet, API failure)

2. Define User Interactions

- Each test case represents real user behaviour such as: Searching for a city's weather, Allowing location access, Viewing forecast details

3. Design Happy Path Test Cases

- Focus on validating correct system behaviour under normal conditions.
- Example: User searches a valid city and weather details are shown

4. Design Error Path Test Cases

- Validate system behaviour under invalid or unexpected conditions.
- Example: User enters invalid city name → error message shown, API failure → graceful error handling message.

5. Break Down Steps and Expected Results

- Each test case includes: Step-by-step actions (e.g., open app → enter city → click search), Expected result (e.g., weather details displayed correctly)
- Ensures clarity for manual testers and automation scripts

6. Use Clear Naming and IDs

- Test cases are uniquely identified for easy tracking: TC01 – Search Valid City Weather, TC02 – Invalid City Search Error Handling
- Helps map test cases to user stories and requirements

7. Separate Test Suites

- o Test cases are grouped based on functionality: Location & Weather Display, Error Handling & Edge Cases
- o Improves organization and test execution flow in Azure DevOps.

8. Prioritize and Review

- o Critical user actions are marked high-priority.
- o Reviewed for completeness and traceability against feature requirements.

1.New test plan

New Test Plan

Name *
Weather Application Test Plan

Area Path *
Weather App

Iteration *
Weather App/Sprint 1 - Basic Weather Functionality 18/02/2026 - 25/02/2026

Create Cancel

2.Test suite

Weather Feature Testing (ID: 16)

Define Execute Chart

Test Cases (7 items)

Title	Order	Test Case Id
Verify weather details for valid city input	1	17
Verify error message for invalid city input	2	18
Verify UI responsiveness and usability	3	20
Verify UI responsiveness and usability	4	21
Verify UI responsiveness and usability	5	22
Verify UI responsiveness and usability	6	23
Verify UI responsiveness and usability	7	24

New Test Case

Test Suites

Filter suites by name

Weather Application Test Plan

Weather Feature Testing (7)

New Suite

- Assign configurations
- Export
- Open
- Assign testers to run all tests
- Rename
- Delete
- Import test suites

3. Test case

Give two test cases for at least five user stories showcasing the happy path and error scenarios in azure DevOps platform.

Weather App – Test Plans

USER STORIES

- As a user, I want to search weather information by entering a city name so that I can know the weather conditions of any location.
- As a user, I want to view the current temperature, humidity, and weather condition so that I can understand the present climate.
- As a user, I want to see a 5-day weather forecast so that I can plan my future activities.
- As a user, I want the application to automatically detect my location so that I receive weather updates without manual input.
- As a user, I want to see an error message when an invalid city name is entered so that I know the input is incorrect.
- As a user, I want a simple and visually appealing interface so that the application is easy to use.

Test Suites

Test Suit: TS01 – Weather Feature Testing (ID: 16)

1. TC01 – Successful Weather Search

- **Action:**
 - Open the Weather App.
 - Enter a valid city name (e.g., Chennai).
 - Click "Search".
- **Expected Results:**
 - Weather App loads successfully.
 - City name is accepted without error.
 - Weather data is fetched from API.
 - Weather information is displayed successfully.
- **Type:** Happy Path

2. TC02 – View Current Weather Details

- **Action:**
 - Open the Weather App.
 - Search for a valid city.
 - Observe the weather details section.
- **Expected Results:**
 - Temperature is displayed correctly.
 - Humidity is shown correctly.
 - Wind speed is displayed.
 - Weather condition (e.g., Clear Sky, Broken Clouds) is shown.
- **Type:** Happy Path

3. TC03 – Display 5-Day Forecast Graph

- o **Action:**
 - Open the Weather App.
 - Search for a valid city.
 - Scroll to the forecast section.
- o **Expected Results:**
 - 5-day forecast bar graph is displayed.
 - Temperature values appear above each bar.
 - Weather conditions appear below each bar.
 - Graph is visually aligned and readable.
- o **Type:** Happy Path

4. TC04 – Automatic Location Detection

- o **Action:**
 - Open the Weather App.
 - Allow browser location permission.
 - Wait for location detection.
- o **Expected Results:**
 - Browser requests location permission.
 - User location is detected successfully.
 - Weather details are displayed automatically for detected location.
- o **Type:** Happy Path

5. TC05 – Invalid City Search Error

- o **Action:**
 - Open the Weather App.
 - Enter an invalid city name (e.g., xyz123).
 - Click "Search".
- o **Expected Results:**
 - Invalid input is processed.
 - Error message such as "**City not found**" is displayed.
 - Application does not crash or freeze.
- o **Type:** Error Scenario

6. TC06 – User Interface Validation

- o **Action:**
 - Open the Weather App.
 - Observe layout and design elements.
 - Interact with buttons and resize window.
- o **Expected Results:**
 - UI loads with proper styling.
 - Buttons are clickable and responsive.

- Layout is clean and visually appealing.
- Application remains usable on resized screen.
- **Type:** UI/Usability Validation

Test Cases

TEST CASE 17

17 Verify weather details for valid city input

Lokshana Senthilkumar

0 Comments Add Tag

Save and Close

Follow

State Closed

Area Weather App

Reason Obsolete

Iteration Weather App\Sprint 1 - Basic Weather Functionality

Updated by Lokshana Senthilkumar: Sunday

Steps Summary Associated Automation 1 0

Steps

Steps	Action	Expected result
1.	Open Weather App	Application loads successfully
2.	Enter valid city name (e.g., Chennai)	City name accepted
3.	Click Search button	Request sent to API
4.	View results	Weather details displayed

Click or type here to add a step

Recent test results

Passed by Lokshana Senthilkumar with Windows 10

Completed 13:15, 26/04/2026

Passed by Lokshana Senthilkumar with Windows 10

Completed 13:11, 26/04/2026

View all test result history

Deployment

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status](#)

TEST CASE 23

23 Verify error message for invalid city

jothi shree

0 Comments Add Tag

Save and Close

Follow

State Design

Area Weather App

Reason New

Iteration Weather App\Sprint 1 - Basic Weather Functionality

Updated by Lokshana Senthilkumar: Sunday

Steps Summary Associated Automation 1 0

Steps

Steps	Action	Expected result
1.	Enter invalid city (xyz123)	Input accepted
2.	Click Search	Request sent
3.	Observe result	Error message shown

Click or type here to add a step

Recent test results

Passed by Lokshana Senthilkumar with Windows 10

Completed 13:40, 26/04/2026

Passed by Lokshana Senthilkumar with Windows 10

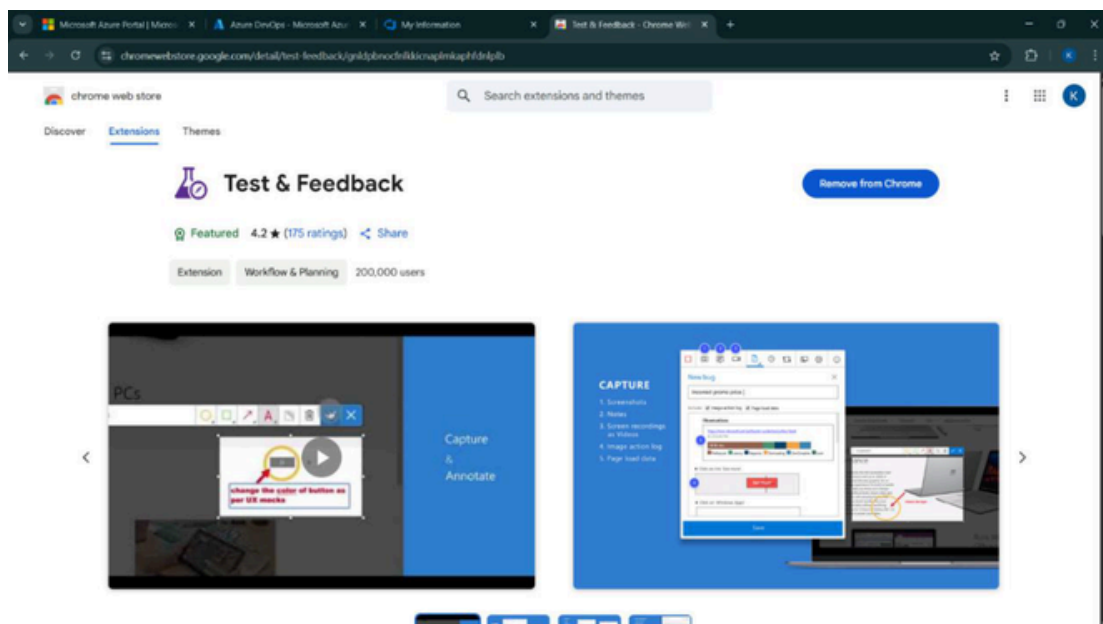
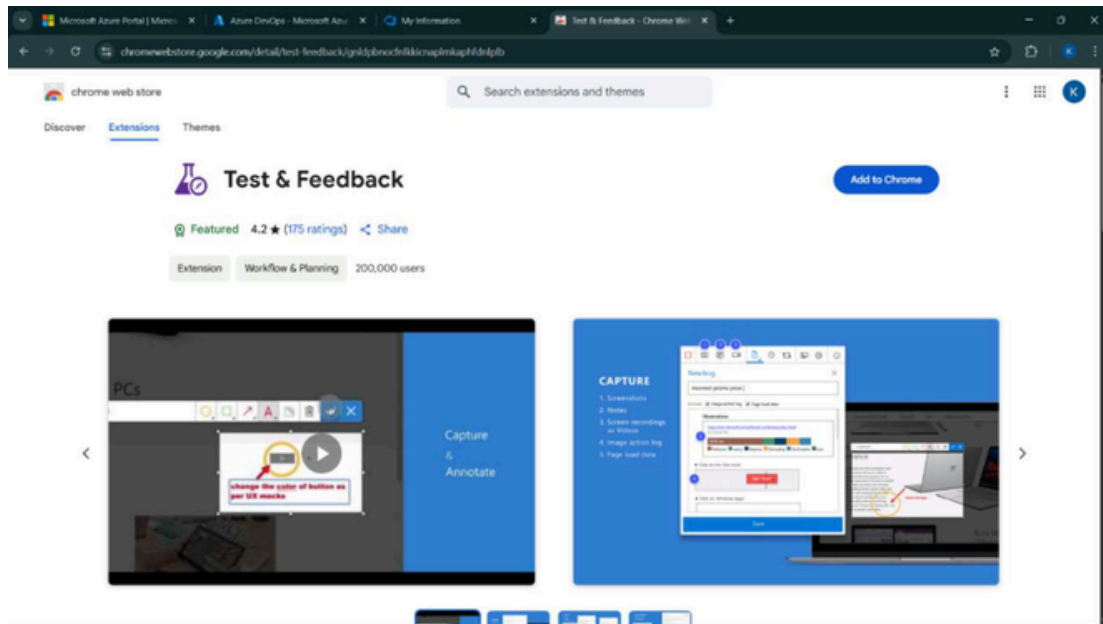
Completed 13:35, 26/04/2026

View all test result history

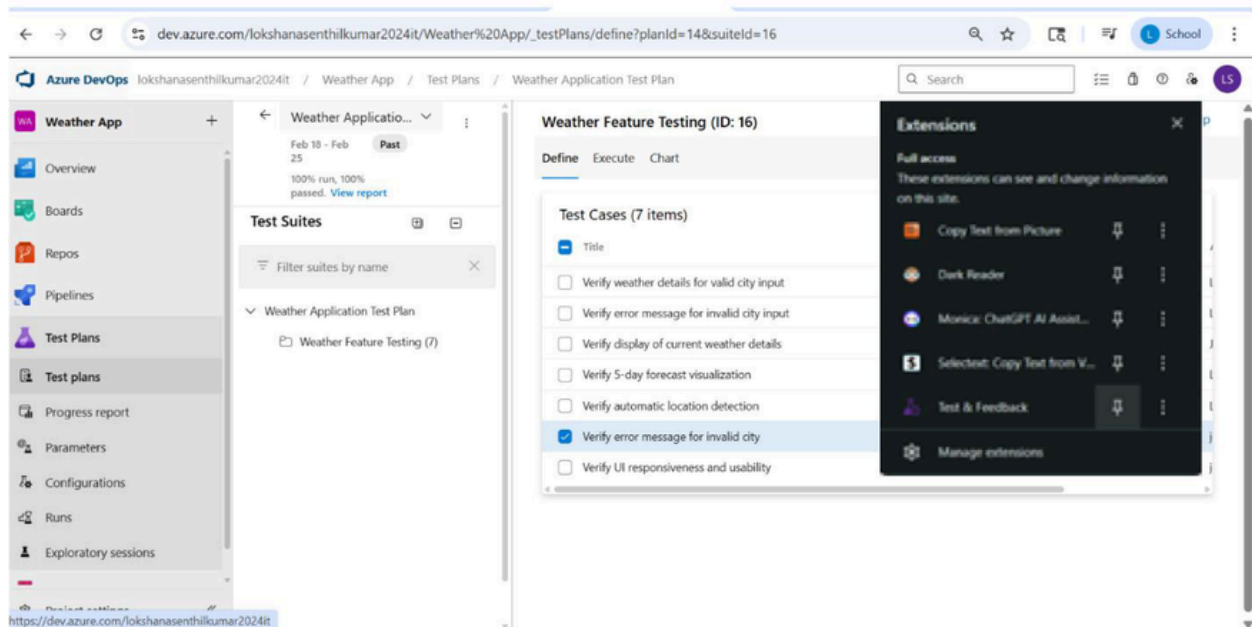
Deployment

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status](#)

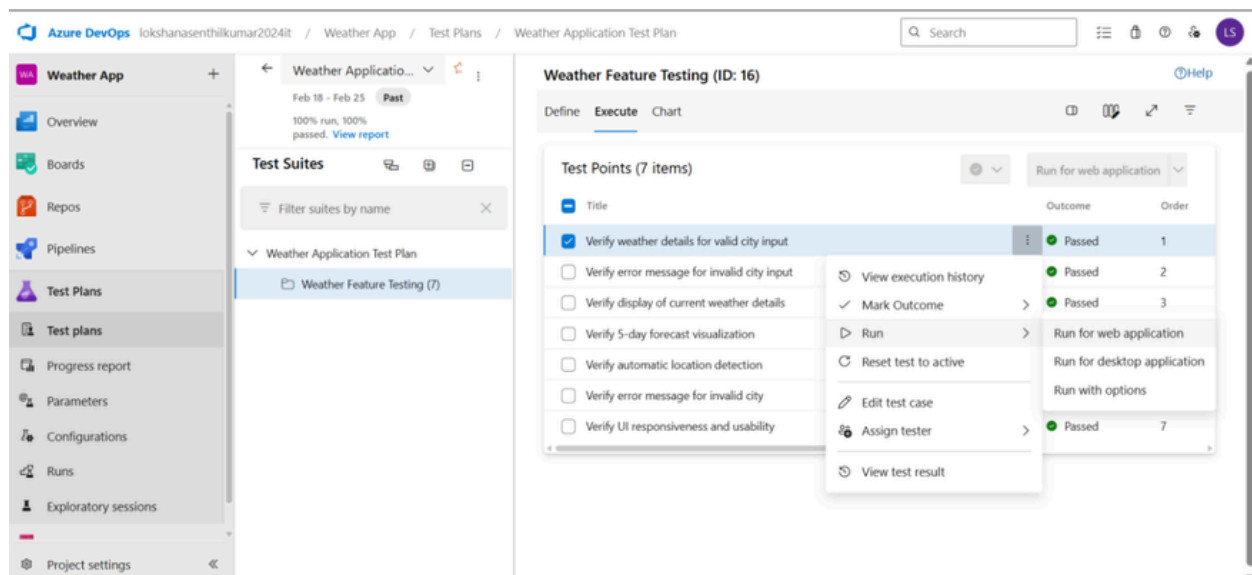
4.Installation of test

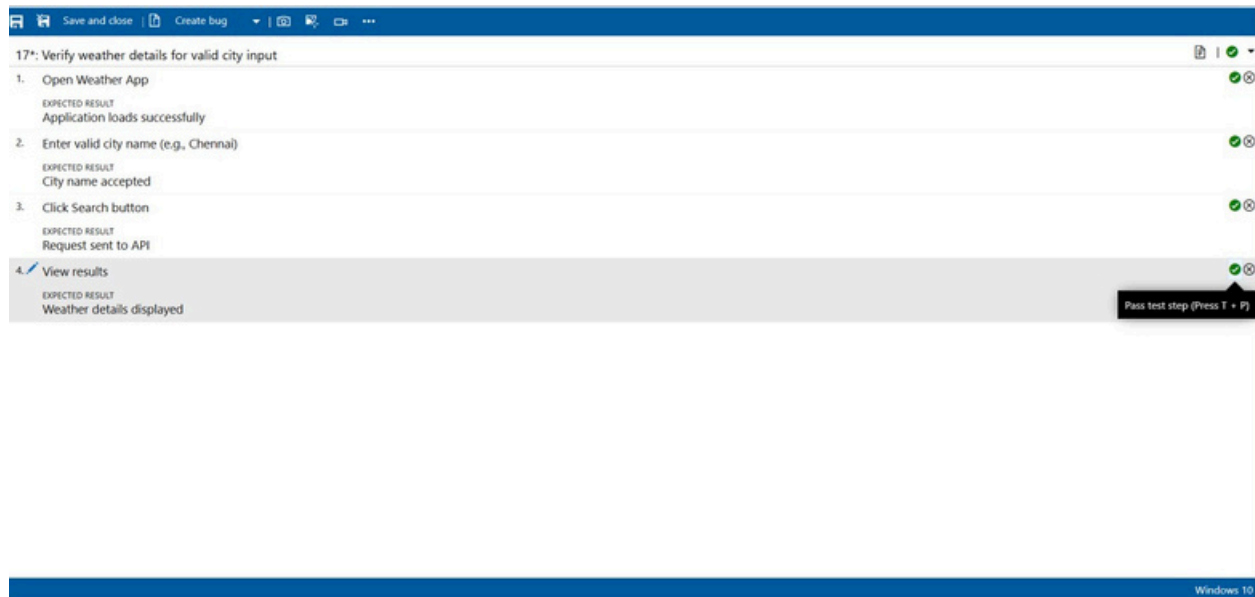


Test and feedback
Showing it as an extension

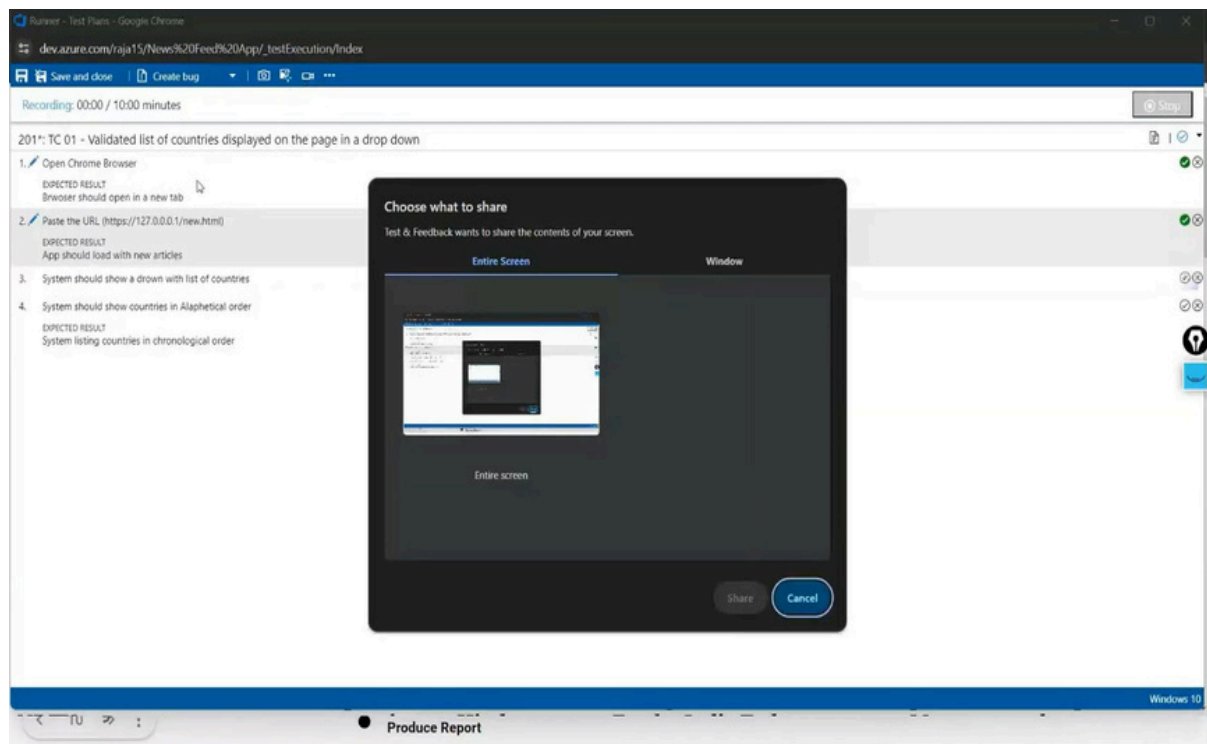


5. Running the test cases

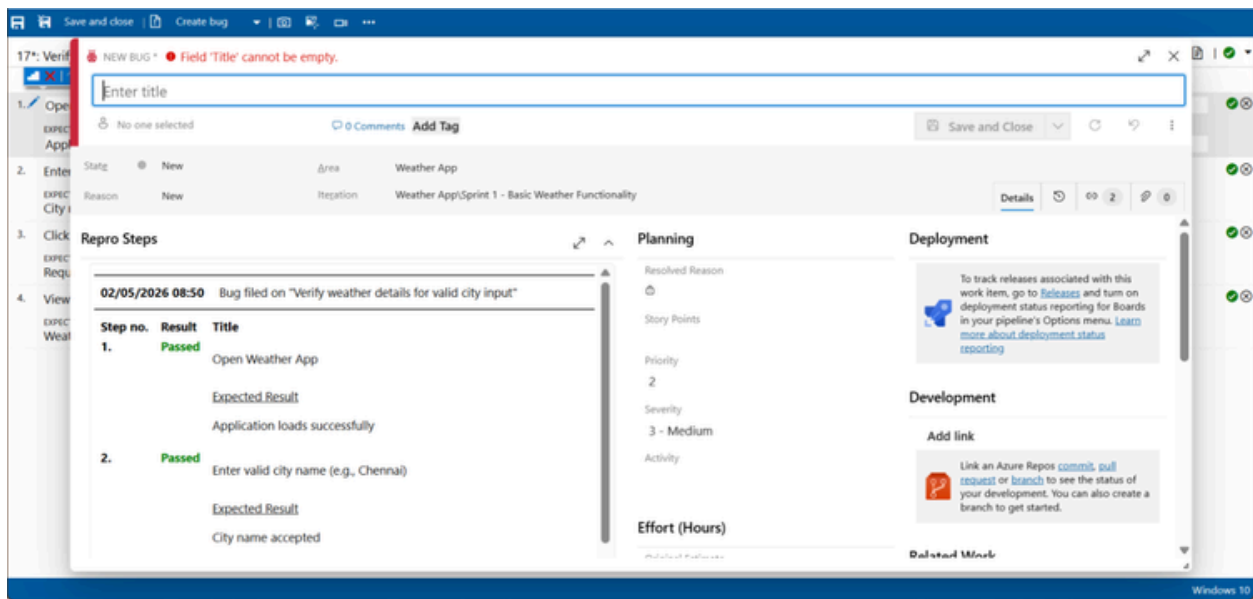
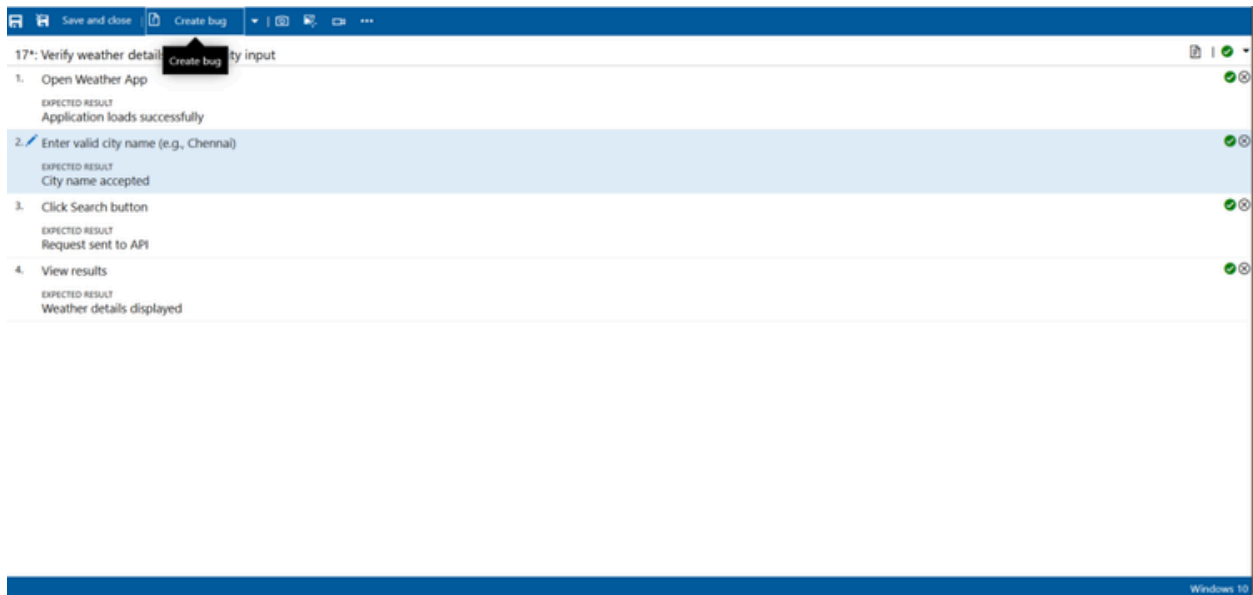




6. Recording the test case



7. Creating the bug



8. Test case results

The screenshot shows the Azure DevOps interface for a test run. The left sidebar lists navigation options like Overview, Boards, Repos, Pipelines, Test Plans, and Runs. The main area displays the test run details for '20 Verify display of current weather' on 'Windows 10'. The test passed, with a start time of 26/04/2026 13:38 and a duration of 6s. A table shows the test steps and their outcomes.

Step	Outcome	Action	Expected Result
1	Passed	Search for a valid city	Weather data fetched
2	Passed	Observe temperature	Temperature displayed
3	Passed	Observe humidity	Humidity displayed
4	Passed	Observe condition	Weather condition shown

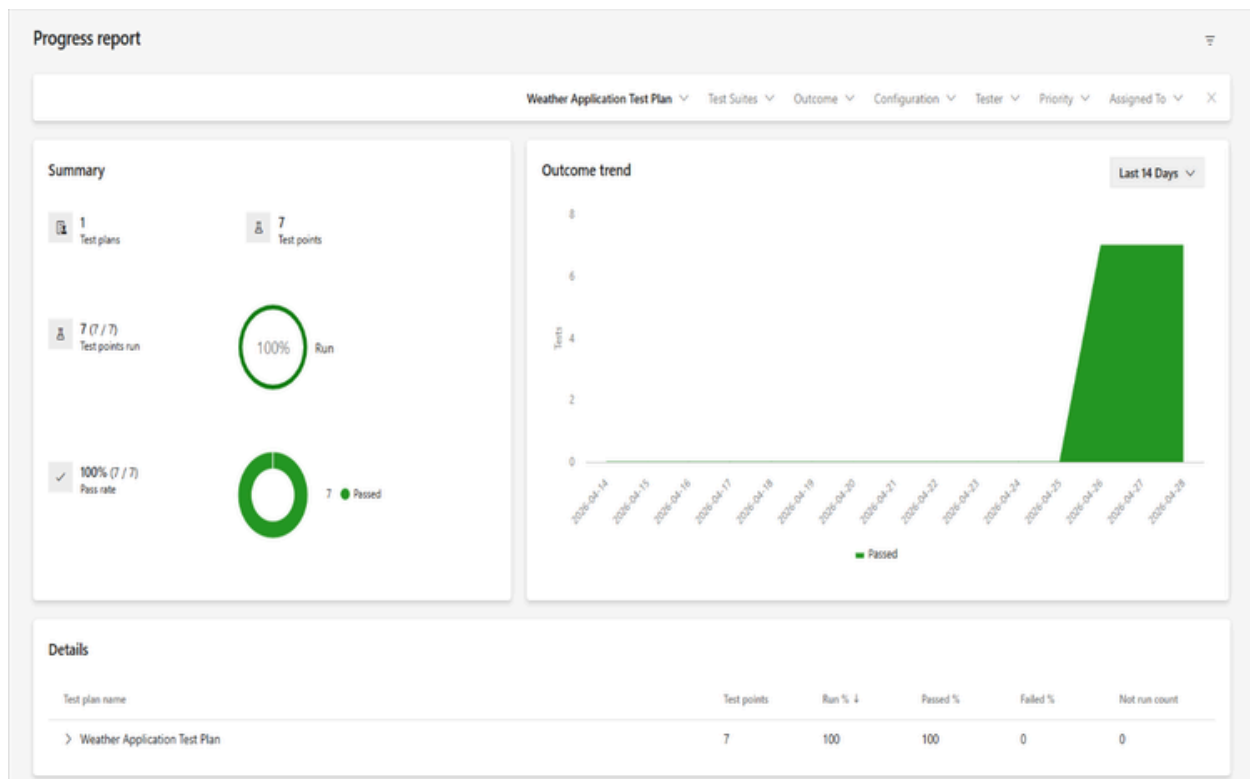
9. Test report summary

- Assigning bug to the developer and changing state

The screenshot shows the Azure DevOps interface with a modal window titled 'Test point ID 1 execution history' open. The modal displays the test case details and a table of execution history for '17 Verify weather details for valid city input'. The test suite is '16 Weather Feature Testing' and the current tester is 'Lokshana Senthikumar' on 'Windows 10' configuration. The table shows four successful test runs.

Outcome	Test run	Run by	Time completed
Passed	9	Lokshana Senthiku...	2m ago
Passed	8	Lokshana Senthiku...	9m ago
Passed	4	Lokshana Senthiku...	Sunday
Passed	2	Lokshana Senthiku...	Sunday

10. Progress report



11. Changing the test template

Organization Settin...
lokshanasenthikumar2024it

Search Settings

General

Overview

Projects

Users

Billing

Global notifications

Usage

Extensions

Microsoft Entra

Security

Security overview

Policies

Permissions

All processes

Help

Filter by process name

Processes

Fields

Name	Description	Team projects
Basic (default)	This template is flexible for any process and great for teams getti...	0
Agile	This template is flexible and will work great for most teams using ...	1
Scrum	This template is for teams who follow the Scrum framework.	0
CMMI	This template is for more formal projects requiring a framework f...	0

12.View the new test case template

The screenshot displays the 'Organization Settings' page in Azure DevOps. The left sidebar contains a 'General' section with links to Overview, Projects, Users, Billing, Global notifications, Usage, Extensions, and Microsoft Entra. Below this is a 'Security' section with a link to Security overview. The main content area is titled 'All processes > Agile' and has tabs for 'Work item types', 'Backlog levels', and 'Projects'. A blue banner at the top of the main area states: 'System processes cannot be customized. To add customization create an inherited process.' Below the tabs, there is a table with two columns: 'Name' and 'Description'. The table contains one entry: 'Weather App' with a description: 'Project Description – Weather Application The Weather Application is a user-friendly web-based syst'. The 'Weather App' entry is highlighted in blue.

Organization Settings
lokshanasenthilkumar2024it

Search Settings

General

- Overview
- Projects
- Users
- Billing
- Global notifications
- Usage
- Extensions
- Microsoft Entra

Security

- Security overview

System processes cannot be customized. To add customization create an inherited process.

All processes > Agile

Work item types Backlog levels Projects

Name	Description
Weather App	Project Description – Weather Application The Weather Application is a user-friendly web-based syst

Result:

The test plans and test cases for the user stories is created in Azure DevOps with Happy Path and Error Path

EXP NO: 9	LOAD TESTING AND PIPELINES
------------------	-----------------------------------

Aim:

To create an Azure Load Testing resource and run a load test to evaluate the performance of a target endpoint and to create and demonstrate an Azure DevOps pipeline for automating application builds, tests, and deployment.

Load Testing

Azure Load Testing:

Azure Load Testing allows you to simulate high traffic and stress tests for your web applications and APIs to understand how they perform under load. It helps identify performance bottlenecks, scalability issues, and optimize resource usage before deployment.

Steps to Create an Azure Load Testing Resource:

Before you run your first test, you need to create the Azure Load Testing resource:

1. Sign in to Azure Portal
Go to <https://portal.azure.com> and log in.
2. Create the Resource
 - o Go to *Create a resource* → Search for “Azure Load Testing”.
 - o Select Azure Load Testing and click Create.
3. Fill in the Configuration Details
 - o *Subscription*: Choose your Azure subscription.
 - o *Resource Group*: Create new or select an existing one.
 - o *Name*: Provide a unique name (no special characters).
 - o *Location*: Choose the region for hosting the resource.
4. (Optional) Configure tags for categorization and billing.
5. Click Review + Create, then Create.
6. Once deployment is complete, click Go to resource.

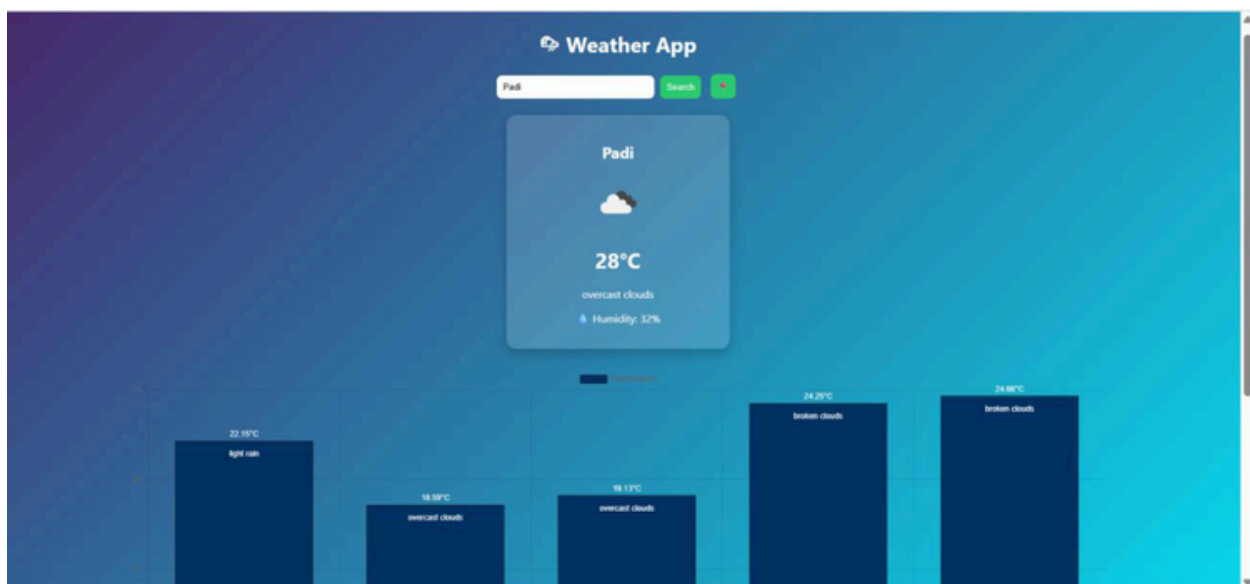
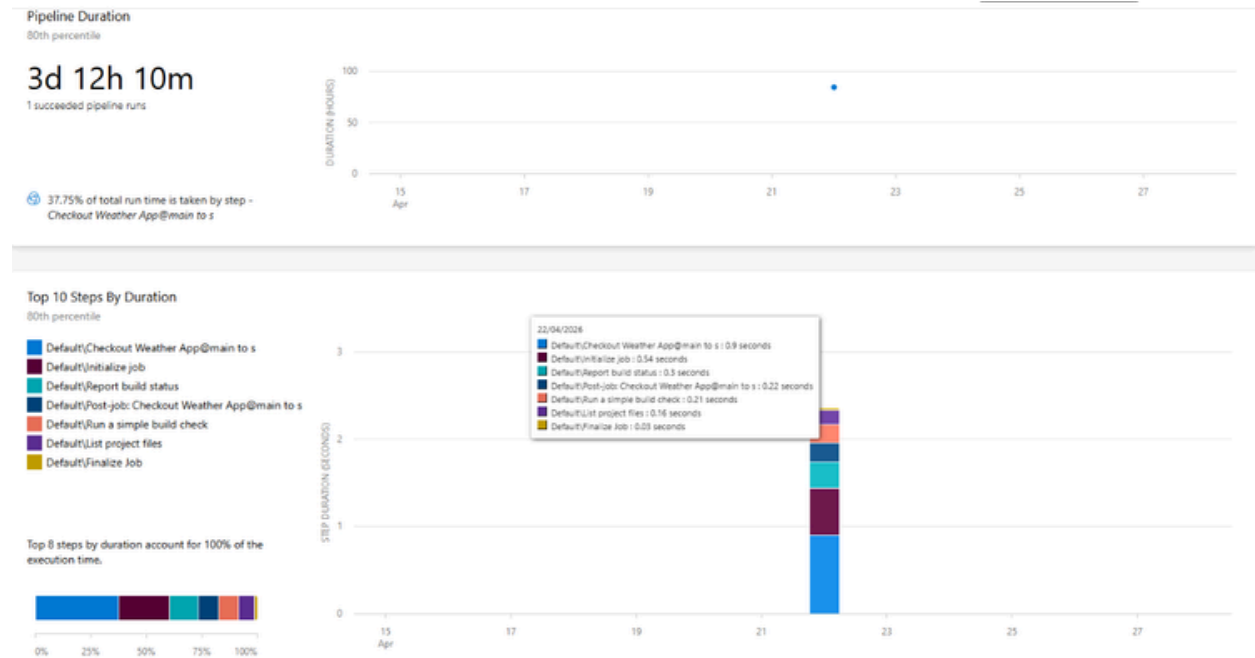
Steps to Create and Run a Load Test:

Once your resource is ready:

1. Go to your Azure Load Testing resource and click Add HTTP requests > Create.
2. Basics Tab
 - o *Test Name*: Provide a unique name.
 - o *Description*: (Optional) Add test purpose.
 - o *Run After Creation*: Keep checked.
3. Load Settings
 - o *Test URL*: Enter the target endpoint (e.g., <https://yourapi.com/products>).

- Click Review + Create → Create to start the test.

Load Testing



Pipelines

Description:

This experiment demonstrates how to connect a GitHub-hosted Weather Application project with Azure DevOps. The pipeline is configured to integrate the project repository, validate project files, and automate basic build verification through Azure Pipelines. This ensures that project changes are tracked efficiently, supports continuous integration practices, and improves reliability and maintainability of the application throughout development.

Steps:

1. Connect GitHub to Azure DevOps:
 - o In Azure DevOps, create a new project.
 - o Create a pipeline and select GitHub as the source.
 - o Authorize access to your GitHub repository, ensuring that Azure DevOps can pull the repository for your pipeline.
2. Create azure-pipelines.yml in Your Repo Root:
 - o In your GitHub repository, create a new file called azure-pipelines.yml in the root directory.
 - o Add the following basic pipeline configuration for HTML, CSS, and JavaScript-based Weather Application:

yml Code

```
#Starter pipeline
#Start with a minimal pipeline that you can customize to build and deploy your code.
#Add steps that build, run tests, deploy, and more:
#https://aka.ms/yaml
```

```
trigger:
- main
```

```
pool:
  vmImage: 'ubuntu-latest'
```

```
steps:
- script: echo "Weather App Build Successful!"
  displayName: 'Run a simple build check'
```

```
- script: |
  echo "Checking files..."
  ls
  displayName: 'List project files'
```

3. Pipeline Tasks Include:
 - o Setting up the pipeline environment using the default Azure-hosted agent (ubuntu-latest).
 - o Validating the Weather Application project files stored in the repository.
 - o Running a simple build verification script to ensure the pipeline is configured correctly.
 - o Listing project files to confirm successful repository integration and pipeline execution.
4. Run and Monitor Pipeline:
 - o Commit changes to the main branch of your repository to trigger the pipeline in Azure DevOps.
 - o Monitor the pipeline execution logs in the Azure DevOps portal to verify build steps, detect errors, and confirm successful execution of the configured tasks.

Pipeline

The screenshot displays the Azure DevOps pipeline interface for a build named '#20260418.1 • Set up CI with Azure Pipelines' for the 'Weather App' project. The build status is 'Succeeded' (indicated by a green checkmark). A message states: 'This run is being retained as one of 3 recent runs by main (Branch).' with a 'View retention leases' link. The interface includes tabs for 'Summary' and 'Code Coverage'. The 'Summary' tab shows the build was triggered by 'Lokshana Senthilkumar' on the 'main' branch of the 'Weather App' repository. It also displays the time started and elapsed (19 Apr at 00:31, 7s), related work items (0), and artifacts (0). A 'Get started' link is provided for tests and coverage. Below the summary, the 'Stages' tab is active, showing a single stage 'Stage' with 1 job completed in 6s. A 'Rerun Stage' button is visible at the bottom of the stage view.

← Jobs in run #20260418.1

Weather App

Jobs

▼ Attempt #2

▼ ✓ Job2s

✓ Initialize job<1s

✓ Checkout Weather App@mai...<1s

✓ Run a simple build check<1s

✓ List project files<1s

✓ Post-job: Checkout Weather A...<1s

✓ Finalize Job<1s

✓ Report build status<1s

Show Previous Attempts

✓ Job

1 Pool: Azure Pipelines

2 Image: ubuntu-latest

3 Agent: Azure Pipelines 1

4 Started: Wed at 12:41

5 Duration: 2s

6

7 ▶ Job preparation parameters

Result:

Successfully created the Azure Load Testing resource and executed a load test to assess the performance of the specified endpoint and also demonstrated pipelines in azure devops.

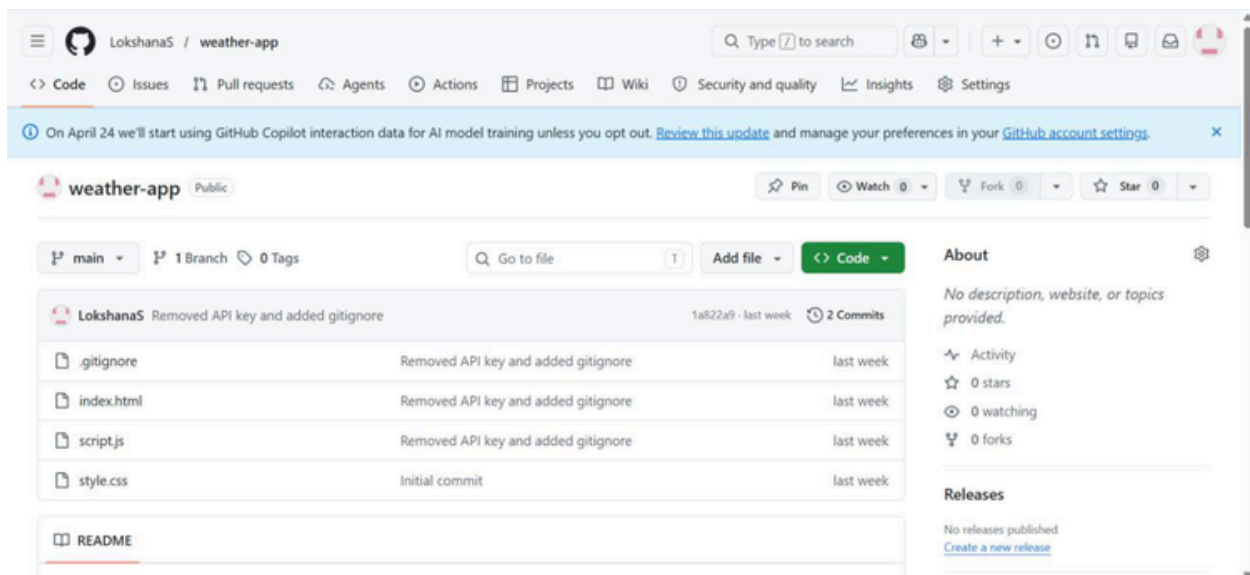
EXP NO: 10

GITHUB: PROJECT STRUCTURE & NAMING CONVENTIONS

Aim:

To provide a clear and organized view of the project's folder structure and file naming conventions, helping contributors and users easily understand, navigate, and extend the Weather App project.

GitHub Project Structure



Result:

The GitHub repository clearly displays the organized project structure and consistent naming conventions, making it easy for users and contributors to understand and navigate the codebase.